

Manual MLOps Challenge

1. Though for the assignment I used just 2 versions of dataset which was not that difficult for me to maintain, but what I feel is if one is having too many versions of dataset which can be due to updating it more number of times when there is drift, then it becomes very difficult to keep track of which data version is currently used and what has created the version. There may also be some other reason for using some other version of dataset. Manifest.txt file keeps track of when the dataset version was created and which file created it and what version was created. This helps in keeping track of how many times I am overwriting the same data by running data_prep.py script multiple times. In fact more information can be stored in manifest.txt for better clarity, but we need to write code manually for logging the specific requirements in manifest.txt
2. I used config.yaml to ensure the production model matches the training model. Both train.py and inference.py read: the same current_version and the same model path. During training the model was saved as models/model_vX.pkl, which was the same as the one API loaded during deployment as the configuration file was the same for both.
3. I removed all the hardcoded file paths from all scripts and put in config.yaml which ensured consistency. Example: raw data path, processed directory, current version, split index all were taken from config.yaml. It means by simply changing the version to v2 in the config file, the [train.py](#) script will use v2_cleaned.csv without changing the script. If the config file was missing, the script would fail immediately, which is actually good because it prevents silent errors. This made the training script not only reproducible but very easy to control.
4. Top 3 features that should be automated in MLOps tool:

I. Data versioning: so that we don't have to take care of manifest.txt. It will automatically enter the relevant info about the version and when it was used and other important info, so that we can keep track of what we are doing more easily.

II. Retraining triggers: whenever there is drift, it will automatically sense it and give retraining triggers, so that we don't have to go and look every time there is a data drift.

III. Like data versioning keeps track of the dataset, model registry keeps track of the model being trained and deployed. It should also be automated so that it is easier for us to know which model is performing worse or better.

5. Sometimes I forget to restart the server after retraining a new model. The API would continue serving the old model even though I believed the new one was live. This caused confusion because monitoring results did not match expectations. I added deployment_log.csv that recorded the model version every time the server started. But manual deployments can make errors and hence automated deployment is more preferable.
6. Model registry is the automated version of Phase B. Phase B required saving model files, metadata JSON, logs and version names. Automated model registry would make this very easy. There would be no mistake and it would have saved time because it will save all the required important files which we may miss. I can simply register and model and promote it to production, by that we will know everything about our model in case there needs to be any change in future. But the manual process helped me understand the complexity behind the automated processes.